Project 4: Out of Control Planning

RRT, KPIECE1 and Reachability-Guided RRT for a pendulum and a car

Darshon Singh (ds161) Winson Lin (wl85)

COMP/ELEC/MECH 450/550 — Fall 2026

Problem: planning with dynamics, $\dot{q} = f(q, u)$

Torque-limited pendulum

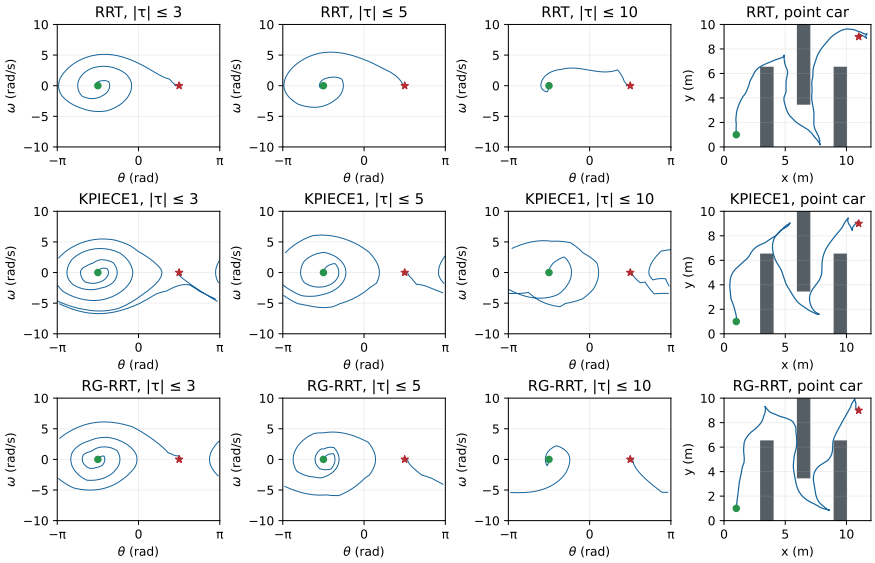
- $q = (\theta, \omega) \in S^1 \times [-10, 10]$
- $\dot{\theta} = \omega,\ \dot{\omega} = -9.81 \cos\theta + \tau$
- $|\tau| \leq 3, 5, 10$; no obstacles
- hanging $(-\pi/2, 0)$ $\rightarrow$ upright $(\pi/2, 0)$

Car in a street

- $q = (x, y, \theta, v) \in SE(2) \times [-2, 2]$
- $(\dot{x}, \dot{y}, \dot{\theta}, \dot{v}) = (v\cos\theta, v\sin\theta, \omega, \dot{v})$
- $|\omega|, |\dot{v}| \leq 1$; reverse allowed; point robot
- $(1, 1) \rightarrow (11, 9)$ through alternating gaps

OMPL `control`: sample control + duration (0.02–0.5 s), integrate with RK4, check validity every 0.02 s. Goals are balls (0.15 pendulum, 0.5 car): exact state hits have negligible probability under continuous random sampling.

Example solutions (all independently re-integrated and checked)



Torque limits change the strategy

- Holding still at θ needs $\tau = 9.81 \cos\theta$.
- $|\tau| \leq 10 > 9.81$: can hold any angle $\Rightarrow$ lift almost directly (2–3 velocity reversals).
- $|\tau| \leq 5$ or 3: cannot hold the bar near horizontal $\Rightarrow$ must *pump energy* by swinging.
- Energy must rise by $2 \cdot 9.81$; $\dot{E} = \tau\omega$. Smaller torque needs more angular travel to supply this work.
- Observed: 7–8 reversals at $\tau = 3$, 3–5 at $\tau = 5$; durations grow as torque shrinks.

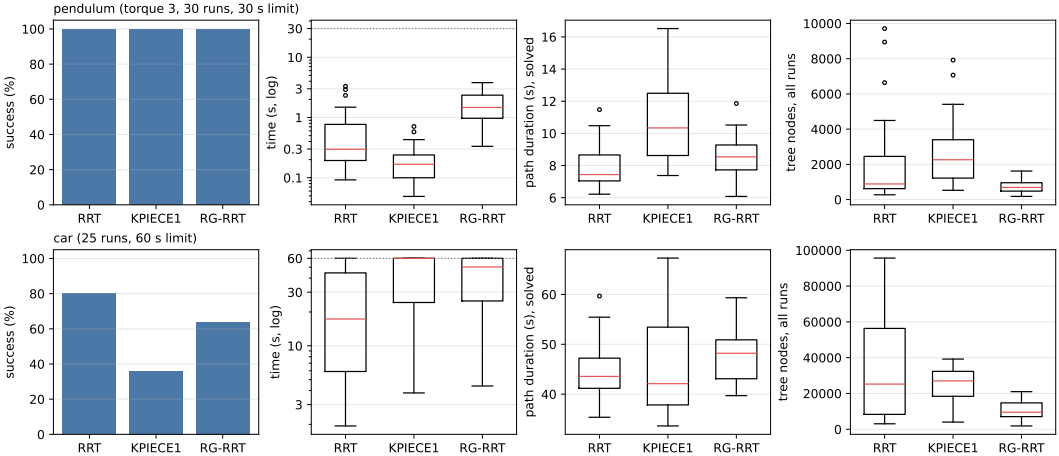
Median over 30 runs	$\tau_{\max} = 3$	$\tau_{\max} = 5$	$\tau_{\max} = 10$
RRT path duration (s)	7.44	4.91	3.39
RRT computation time (s)	0.30	0.46	0.34
KPIECE1 path duration (s)	10.34	8.31	5.12
KPIECE1 computation time (s)	0.17	0.17	0.09
RG-RRT path duration (s)	8.54	5.72	3.32
RG-RRT computation time (s)	1.47	1.38	1.86

RG-RRT: filter samples using approximate local reachability

1. Every node stores $R(q)$: end states of k evenly spaced controls applied for s steps (`propagateWhileValid`; invalid $\Rightarrow$ not reachable).
2. Sample q_{rand}, find nearest q_{near}.
3. If no $r \in R(q_{\text{near}})$ is closer to q_{rand} than q_{near}: **discard** and resample.
4. Else extend q_{near} towards q_{rand} as in RRT; compute R of the new node.

Assumptions: bounded `RealVectorControlSpace`; k controls span u_0 (torque / steering rate), other inputs at the centre (car $\dot{v} = 0$). Default $k = 11$, $s = 5$ (0.1 s).

Benchmark: RRT vs. KPIECE1 vs. RG-RRT



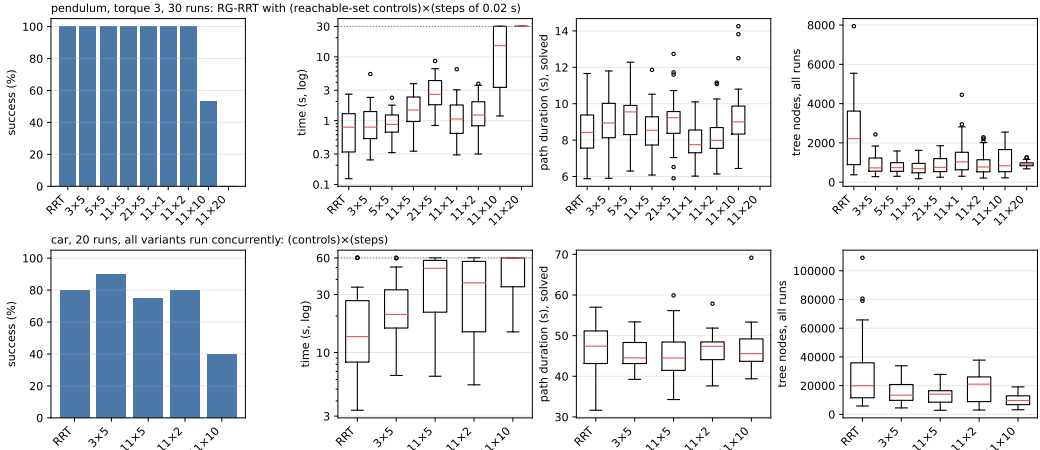
Benchmark summary

System / planner	runs	success	time (s): median, mean		duration (s)	tree nodes
Pendulum RRT	30	100%	0.30	0.67	7.44	888
Pendulum KPIECE1	30	100%	0.17	0.20	10.34	2262
Pendulum RG-RRT (11 ctrl, 5 steps)	30	100%	1.47	1.73	8.54	690
Car RRT	25	80%	17.32	25.03	43.56	25237
Car KPIECE1	25	36%	60.02	45.31	42.10	27029
Car RG-RRT (11 ctrl, 5 steps)	25	64%	50.28	41.33	48.20	9512

30 pendulum / 25 car runs per planner; 30 / 60 s limits. Concurrent runs: CPU load can vary.

- Pendulum: everyone solves 100%; KPIECE1 fastest (projection = full state), but longest trajectories.
- Car: RRT best (80%, median 17 s); RG-RRT 64%; KPIECE1 36% (projection has roughly 3800 possible cells).
- Default RG-RRT builds the smallest median tree here (car: 9.5k vs. 25k nodes).

RG-RRT trade-offs: number of controls and time period



Head-to-head conclusions

- **Buys:** 1.3–3× smaller trees; 50–80% of samples rejected by the reachable-set test.
- **Costs:** $k \cdot s$ extra propagation steps per node (up to 55 vs. ≈13 for an RRT extension) ⇒ slower in wall-clock time than RRT on both systems.
- **Controls k:** benefit saturates at $k = 3$ (bang–zero–bang); time grows linearly (pendulum $0.8\,\mathrm{s} \to 2.6\,\mathrm{s}$ for $k = 3 \to 21$). Car 3×5: best variant, 90% success.
- **Period s:** 1–2 steps cheap and good; 10 steps ⇒ 53% / 40% success, 20 steps ⇒ 0% on pendulum: $R(q)$ overshoots, nearby (and goal) samples get discarded, no fine corrections.
- **Verdict:** keep reachable sets cheap and local; for these systems plain RRT is still faster.

What went well, what was hard

- **Went well:** independent SciPy replay of every saved control path (bounds, collisions, goal) validated the models and saved paths; one shared planning/benchmark layer for both systems.
- **Hard:** tuning goal radii, propagation step and control durations so the planners succeed reliably; choosing the reachable-set duration — too short and $R(q)$ is indistinguishable from q, too long and it overshoots.
- **Takeaway:** reachability guidance trades extra propagation per node for fewer, better-directed nodes; whether that pays off depends on the system and on k and s.